Pingie Documentation

AI support bots that answer from your docs. Guides, how Pingie works, and the API.

Getting started

Introduction

Pingie gives your website an AI support assistant that answers only from your own documentation, and hands the conversation to a person when it can't.

Pingie is a support chatbot you train on your own content: help-center pages, PDFs, policies, product docs. Visitors ask questions in a small chat widget on your site. Pingie answers from your sources, shows which document each answer came from, and admits when it doesn't know instead of making something up.

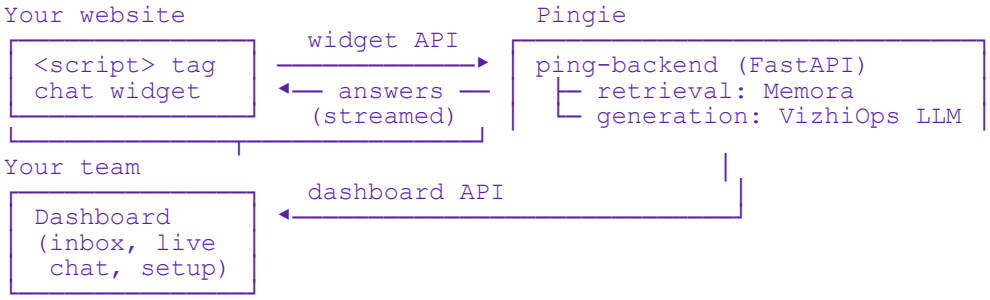
When the bot can't answer, your team takes over. Visitors can ask for a person, an agent replies live from the dashboard, and the answer can be taught back to the bot so the same question is answered next time.

Quickstart →Go from sign-in to a live bot on your site in about ten minutes. **Install the widget** →One script tag, allowed domains, and going live. **How answers work** →Retrieval, the similarity floor, citations and fallbacks. **API reference** →Every REST endpoint, generated from the backend.

What you can do with Pingie

Area	What it gives you
Knowledge	Crawl a sitemap (up to 200 pages), add single URLs, or upload PDF, DOCX, TXT, MD and CSV files up to 25 MB. Test retrieval before going live.
Widget	A lightweight chat widget (~10 kB gzipped) that streams answers with source chips. Change the colours, text, launcher icon, position and theme.
Inbox	Every conversation with outcome badges, filters for unanswered, disliked and handed-off chats, search, and CSV export.
Live chat	"Talk to a person" when the bot can't answer, agent presence, browser alerts, and taking over any chat.
Teach the bot	Turn an unanswered question into a saved answer the bot uses from then on.
Analytics	Conversations, resolution rate, first-response time, thumbs-down rate, knowledge gaps and top questions.
Teams	Invite editors and viewers to a workspace, or share a single bot with someone outside it.

How the pieces fit



See Architecture for the full picture.

Where to go next

- New here? Start with the Quickstart, then read Core concepts.
- Embedding on a site? Install the widget and look at the JavaScript API.
- Building an integration? Read the API overview and the API reference.

Quickstart

Create a bot, give it knowledge, check its answers, and put it on your website.

1. Sign in

Open the Pingie dashboard and choose **Continue with Google**. Your first sign-in creates a personal **workspace** on the Free plan. You can create bots right away.

2. Create a bot

On **Bots**, click **New bot**. The wizard has three steps:

1. **Basics.** Enter a name (visitors see it in the widget header) and your website, for example `help.acme.com`. The website is added to the bot's allowed domains automatically.
2. **Knowledge.** Choose one:
   - **Crawl a sitemap.** Pingie suggests `https://<your site>/sitemap.xml`.
   - **Upload files.** PDF, DOCX, TXT, MD or CSV.
   - **Add sources later.**
3. **Launch.** Review and create. The bot starts as a **draft**.

3. Wait for indexing

Open **Knowledge** → **Sources**. Each source moves from *queued* to *indexing* to *ready*, and the table shows progress and chunk counts. Sitemap crawls fetch up to 200 pages, four at a time, and respect `robots.txt`.

Tip *Answers are only as good as the sources. A focused set of help articles beats an entire marketing site.*

4. Test before you go live

Open **Knowledge** → **Test retrieval** and ask a real customer question. You'll see:

- the **standalone query** Pingie searched for,
- the **retrieved chunks** with similarity and rerank scores,
- the **generated answer** with its citations, and timings.

If the right passage isn't retrieved, add or improve a source. If good passages score below the threshold, lower the **similarity threshold** in **Bot settings**. The default is 0.30.

5. Make it yours

- **Widget** → **Appearance.** Title, welcome message, placeholder, up to four suggested questions, accent colour, dark or light theme, launcher icon and position. The preview updates as you type.
- **Manage** → **Bot settings.** Instructions, tone (friendly, professional or concise), reply language, model tier, fallback message and handoff email.

6. Install the widget

Copy the snippet from **Widget** → **Install & embed** and paste it before `</body>` on every page where you want the chat:

```
<script
src="https://cdn.pingie.app/widget.js"
data-bot-id="bot_xxxxxxx"
defer
></script>
```

While the bot is a draft, the widget only loads on `localhost`, and only after you turn on **Allow localhost** in **Widget** → **Allowed domains**. Press **Check again** on the Install page to confirm the widget was seen.

7. Go live

In **Bot settings**, change the status to **Live**. The widget now loads on every hostname in **Allowed domains**. Make sure each one is listed: `help.acme.com`, or `*.acme.com` for all subdomains.

Next steps

- Turn on live chat so a person can step in when the bot can't answer.
- Invite teammates in Teams & sharing.

- Check what your plan includes in Plans & limits.

Core concepts

The objects Pingie works with and the words used across the dashboard and API.

Workspace

A **workspace** (called an *organization* in the API) owns bots and holds the plan and its limits. Everyone gets a personal workspace on first sign-in. You can also be a member of workspaces other people invited you to, and switch between them from the top bar.

Each member has a **role**:

Role	Can do
owner	Everything, including deleting bots, clearing history, renaming the workspace and managing members and sharing.
editor	Change bots: settings, knowledge, appearance and domains. Reply in live chat. Create bots in the workspace.
viewer	Read only: overview, conversations, sources and test retrieval.

Bot

A **bot** is one assistant with its own sources, settings, appearance and allowed domains. Its public ID looks like `bot_7f3k2a9`, and you put it in the widget snippet.

A bot has a **status**:

Status	Behaviour
draft	The widget loads only on localhost, and only with Allow localhost on. Use it to try things out.
live	The widget loads on the allowed domains and answers visitors.
paused	The widget shows Offline and every message gets the bot's offline message.

Source

A **source** is a piece of knowledge: an uploaded file, a single web page, or a sitemap (which becomes many page sources). Sources are chunked and embedded for retrieval. Their status goes `queued` → `indexing` → `ready`, or ends as `failed` with an error.

Team answers is a special text source that holds the answers your team taught the bot from the inbox. See Live chat.

Conversation, message and visitor

- A **visitor** is an anonymous person on your site. The widget remembers them with an ID in `localStorage`, and they appear as "Visitor #4821" until they share an email or you call `Pingie.identify()`.
- A **conversation** is one chat. It has a **status** that says who is answering: `bot`, `waiting` (the visitor asked for a person), `human` (an agent is replying) or `closed`.
- A **message** has a **role**: `visitor`, `bot`, `agent` (a person on your team) or `system` (a short note such as "Zara joined the chat").

Outcome

Every bot reply gets an **outcome**:

Outcome	Meaning
answered	Answered from your sources, with citations.
partial	Part of the question was answered; the rest wasn't in the sources.
fallback	Nothing relevant was found, so the bot said it doesn't know.
smalltalk	A greeting or thanks, answered without searching.
lead	The visitor left their email after a fallback.

Outcome	Meaning
offline	The bot is paused.

A conversation's outcome is the **worst** answer outcome in it (`answered` < `partial` < `fallback`). The inbox's *Unanswered* filter is conversations whose outcome is `fallback`.

Lead and handoff

When the bot can't answer, it can ask for the visitor's email. That email becomes a **lead**: it's saved with the unanswered question and emailed to the bot's **handoff email**. With live chat, the visitor can instead ask to talk to a person straight away.

Model tier

Each bot uses a **model tier** for writing answers: `fast` (lowest latency), `balanced` (the default) or `best` (highest quality). Which tiers you can pick depends on your plan.

Guides

Knowledge sources

Give a bot something to answer from (files, pages and sitemaps), keep it fresh, and check what it retrieves.

A bot answers **only** from its sources. Anything that isn't in them, it won't make up. It falls back instead.

Add sources

Open **Knowledge** → **Sources**.

Upload files

Drag files onto the drop zone or click to choose. Supported types are **PDF, DOCX, TXT, MD and CSV**, up to **25 MB** each. Files go straight to the retrieval engine, which extracts the text, chunks it and embeds it.

Add a URL

Choose **URL**, paste a page address, and Pingie fetches that one page, converts it to clean text, and indexes it.

Crawl a sitemap

Choose **Sitemap** and paste a `sitemap.xml` address (sitemap index files work too). Pingie:

- fetches up to **200 pages**, four at a time, with a 15-second timeout and a 5 MB limit per page,
- follows `robots.txt` and identifies itself as `PingieBot/1.0`,
- refuses private and loopback addresses to prevent server-side request forgery,
- adds each page as a child of the sitemap source, so you can track progress per page.

Note *Each top-level source (a file, a URL or a sitemap) counts against your plan's* ***sources per bot*** *limit. Pages inside a sitemap don't count separately.*

Source status

Status	Meaning
queued	Waiting to be fetched (URLs and sitemaps).
indexing	Being extracted, chunked and embedded. The page refreshes progress on its own.
ready	Searchable. The table shows the chunk count.
failed	Something went wrong; the error is shown on the row. Fix it and choose Re-sync.

Keep sources fresh

- **Re-sync** fetches a URL or sitemap again (or re-extracts a file) and replaces the indexed version. Do this after you change a page.
- **Delete** removes the source and its chunks from retrieval straight away.

Test retrieval

Knowledge → **Test retrieval** runs the same search the bot runs, without saving anything:

1. Enter a question, and optionally change **Top K** (how many chunks to fetch) and the **threshold**.
2. See the rewritten **standalone query**, the ranked **chunks** with similarity and rerank scores, the **answer** that would be sent, and the **timings** for each stage.

If the right passage isn't in the results, the bot can't use it. Add a source that states it plainly. If the right passage is there but below the threshold, consider lowering the bot's **similarity threshold** (Bot settings, default 0.30).

Team answers

Answers your team writes with **Teach the bot** (from an unanswered chat) or **Add answer** (on the Sources page) are stored together in one text source called **Team answers**. Every change rebuilds and re-indexes it, and it counts as a single source for your plan. See Teach the bot.

Writing sources that answer well

- One topic per page or section, with a clear heading. The heading is often what gets matched.
- State facts plainly: "Refunds are available within 30 days of purchase", not "We're flexible on refunds!".
- Keep prices, limits and policies in text, not only in images.
- Remove outdated pages. Pingie can't tell which of two contradicting pages is current.

Install the widget

Add the chat to your site with one script tag, control where it may load, and customise how it looks.

The snippet

Copy it from **Widget → Install & embed** and paste it before `</body>`:

```
<script
src="https://cdn.pingie.app/widget.js"
data-bot-id="bot_xxxxxxx"
defer
></script>
```

Attribute	Required	Purpose
data-bot-id	Yes	The bot's public ID, shown on the Install page.
data-api	No	The Pingie API origin. Set it only when self-hosting, e.g. https://api.example.com.
data-open="true"	No	Open the chat as soon as the page loads.

The script is one file of about 10 kB gzipped with no dependencies. It renders inside a **Shadow DOM**, so your site's CSS can't affect it and its CSS can't leak into your page. If something goes wrong (unknown bot, domain not allowed, API unreachable), it stays hidden and logs one console warning. It never breaks your page.

Single-page apps

Load the script once in your app shell. The widget survives route changes. To control it from your code, use the JavaScript API.

Allowed domains

The widget only loads on hostnames listed under **Widget → Allowed domains**. The browser's `Origin` header is checked on every request.

Entry	Matches
help.acme.com	Exactly help.acme.com.
*.acme.com	acme.com and every subdomain, such as www.acme.com and docs.acme.com.

- Enter hostnames only. A scheme, path or port is stripped if you paste one.
- **Allow localhost** lets the widget run on `localhost` and `127.0.0.1` for testing. Draft bots work only there.
- The list shows when each domain last loaded the widget, so you can see which sites it's actually running on.

Warning *If the widget doesn't appear on your live site, check that the bot is **Live**, that the exact hostname (including `www.` if used) is on the list, and look for a `[Pingie]` warning in the browser console.*

Check the install

On **Install & embed**, press **Check again**. Pingie reports the most recent hostname it saw the widget on, and when.

Appearance

Widget → Appearance changes everything visitors see, with a live preview:

Group	Options
Content	**Header title, welcome message, input placeholder, up to 4 suggested questions (shown as tappable chips until the visitor sends a message).**
Style	**Accent colour (presets or any #RRGGBB), dark or light theme, launcher icon (chat, spark or help), bottom-left or bottom-right.**
Behaviour	**Show cited sources under answers; show the "Powered by Pingie" badge.**

The badge can only be turned off on plans that include **remove branding**. On the Free plan it's always shown.

What visitors get

- **Streamed answers** that appear word by word, with Markdown formatting (escaped for safety).
- **Source chips** under answers; web-page sources link to the page.
- **Thumbs up / down** on every answer. Thumbs-down chats appear under the inbox's *Disliked* filter.
- **Email capture** when the bot can't answer and no agent is available.
- **Talk to a person** when someone on your team is available. See Live chat.
- **Memory across pages**: the conversation is kept in `localStorage` for 7 days, so it survives navigation.
- **Mobile layout**: full screen on screens 480 px wide or less. Esc closes the chat.

Widget JavaScript API

Open the chat, ask questions and attribute conversations to signed-in users from your own code.

Once the script loads, it exposes `window.Pingie`. Calls made **before** the widget has finished loading are queued and replayed, so you can call them straight away.

```
Pingie.open();                      // open the chat panel
Pingie.close();                     // close it
Pingie.toggle();                    // open if closed, close if open

Pingie.ask("How do refunds work?"); // open and send this question as the visitor

Pingie.identify({                   // attribute the chat to a known user
email: "jane@acme.com",
name: "Jane Doe",
});

Pingie.reset();                     // forget this conversation and start a new one
Pingie.destroy();                   // remove the widget from the page

Pingie.version;                     // e.g. "1.0.0"
```

`Pingie.identify({ email, name })`

Use this when the visitor is signed in to your product. The conversation then shows their name and email in the inbox instead of "Visitor #N", and the bot won't ask for an email it already has.

- Both fields are optional; an invalid email is ignored with a console warning.
- It's safe to call before the visitor has said anything: it's applied when the conversation starts.

```
<script>
// after your app knows who the user is
Pingie.identify({ email: currentUser.email, name: currentUser.fullName });
</script>
```

`Pingie.ask(text)`

Opens the chat and sends `text` as the visitor's message. It's handy for "Ask about this" buttons:

```
<button onclick="Pingie.ask('What does the Pro plan include?')">Ask about Pro</button>
```

`Pingie.reset()`

Clears the stored conversation and starts fresh. Call it when a user signs out of your product, so the next person on the same browser doesn't see their chat.

Examples

Open the chat from your own button

```
<a href="#" onclick="Pingie.open(); return false;">Chat with support</a>
```

Open automatically on a help page

```
<script src="https://cdn.pingie.app/widget.js" data-bot-id="bot_xxxxxxx"
data-open="true" defer></script>
```

React

```
declare global {
interface Window { Pingie?: { open(): void; ask(t: string): void; identify(i: { email?:
string; name?: string }): void } }
}

export function HelpButton() {
return <button onClick={() => window.Pingie?.ask("I need help with billing")}>Get
help</button>;
}
```

Note *Building your own chat interface instead of using the widget? See Build a custom chat UI, which uses the same public widget API.*

Conversations & leads

Review every chat, find what the bot couldn't answer, export transcripts, and follow up on leads.

Open a bot and go to **Conversations**. The inbox lists every chat, newest first, 50 at a time, with **Load more** at the bottom.

Filters

Filter	Shows
All	Every conversation.
Live	Chats where a visitor is waiting for a person, or an agent is replying. Waiting chats come first. This filter opens by default when someone is waiting.
Unanswered	Conversations where at least one answer was a fallback.
Disliked	Conversations with at least one thumbs-down.
Handoffs	Conversations where the visitor left an email (a lead).

The search box matches message text, visitor name and visitor email.

Reading a transcript

Click a conversation to see:

- the visitor, when it started, how long it lasted, the page it started on and the visitor's location (from your CDN's geo headers when available),
- every message with its time, the bot's **outcome badge** (partial, fallback, lead…), the **sources it cited** and the visitor's **feedback**,
- system notes such as "Zara from the team joined the chat".

Opening a conversation marks it as read; unread chats show a dot in the list. The list refreshes every few seconds.

Act on unanswered questions

Under each fallback or thumbs-down answer, **Teach the bot the right answer** lets you write the correct answer once. The bot uses it from then on. See Teach the bot.

You can also **Take over** any conversation and reply as a person. See Live chat.

Leads

When the bot can't answer and no agent is available, it asks the visitor for their email. The next message is checked for one, and if there is one, Pingie:

1. saves a **lead** with the email and the question that went unanswered,
2. shows the conversation under **Handoffs**, with the email at the top of the transcript,
3. emails the bot's **handoff email** (Bot settings) with the question; the visitor's address is set as Reply-To, so you can answer by replying.

Note *Emails are sent only when SMTP is configured on the server. Otherwise the lead is still saved and visible in the inbox.*

Export

Export CSV downloads every message of every conversation for the bot, one row per message, with these columns: `conversation_id`, `started_at`, `visitor`, `page`, `role`, `message`, `outcome`, `feedback`, `sources` and `sent_at`.

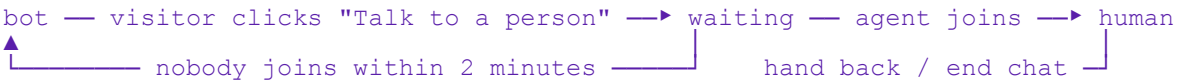
Clear history

Owners can delete all conversations, visitors and leads for a bot from **Bot settings** → **Danger zone** → **Clear history**. This can't be undone.

Live chat & handoff

Let a person take over when the bot can't answer, get alerted when visitors are waiting, and teach the bot the answer for next time.

How handoff works



1. The bot can't answer a question (a **fallback**).
2. If someone on your team is **available**, the widget shows a **Talk to a person** button under the reply. If nobody is, the bot asks for the visitor's email instead (see Leads).
3. The visitor clicks it. The conversation becomes **waiting**, and the widget says *Connecting you to a person…*.
4. Every available agent is alerted. The first to reply or press **Join chat** takes it, and the conversation becomes **human**.
5. While a person is on the chat, the bot stays quiet. The visitor's messages go only to the agent, and the widget header shows *Zara from the team*.
6. When done, the agent chooses **Hand back to bot** (the bot answers again) or **End chat**. If the visitor writes after the chat ended, the bot picks up as usual.

If nobody joins within **2 minutes**, the widget apologises and asks for the visitor's email. A lead is saved, and the bot's handoff email gets a *Missed live chat* message.

Be available

In the dashboard top bar, open **Live** and switch on **Available for live chat**.

- Allow notifications when the browser asks, so you hear and see new chats even in another tab.
- Keep a dashboard tab open while you're available. The tab checks in every 30 seconds, and you count as away about a minute after the last check-in.
- Anyone who can edit the bot can take its chats: workspace **owners and editors**, and people the bot is **shared with as editor**. Viewers can read but not reply.

Alerts

When a visitor starts waiting, every available agent gets:

- a **browser notification** and a short **chime**,
- an amber **Live** button in the top bar with the count. Its menu lists waiting chats and *your chats*,
- **N waiting** on the bot's *Conversations* item in the sidebar,
- the count in the browser tab title, e.g. `(2) Conversations · Pingie`.

Click any of them to open the chat.

Reply

In **Conversations**, the **Live** filter shows waiting and active chats. Open one and type in the reply box at the bottom: **Enter** sends and **Shift+Enter** adds a new line. Your reply reaches the widget within about three seconds.

You can also **Take over** any conversation, even one the bot is handling well, for example to help a high-value customer. Sending a reply takes over automatically.

Action	Effect
Join chat / Take over	You're assigned; the bot stops answering; the visitor sees "Your name from the team joined the chat".
Hand back to bot	The bot answers again; the visitor sees "You're chatting with the assistant again".
End chat	The chat closes; the visitor's next message starts with the bot again.

Note *Agent replies don't count against your plan's monthly message limit. Visitor messages do, as usual.*

Teach the bot

Under every fallback or thumbs-down answer in a transcript there's a **Teach the bot the right answer** link:

1. The visitor's question is filled in; adjust it if needed.
2. Write the answer the way you want the bot to say it.
3. Optionally tick **Also send this answer to the visitor now**. It's ticked by default during a live chat.
4. Save.

The answer is added to the bot's **Team answers** source and re-indexed, so the bot uses it within a minute or so. You can review, add and delete team answers on **Knowledge** → **Sources**. All of them together count as one source on your plan.

How it works underneath

Both the widget and the dashboard **poll** for updates: every 3 seconds while a person is on the chat and the panel is open, and less often otherwise. Polling pauses while the tab is hidden. There's no websocket server to run, and it works with any number of API processes. For the endpoints involved, see Build a custom chat UI and the Conversations API.

Teams & sharing

Invite teammates into a workspace, share a single bot with someone outside it, and control what each person can do.

There are two ways to give someone access:

	Workspace member	Shared bot
Access to	Every bot in the workspace	One bot
Where it shows for them	A workspace in their top-bar switcher	Shared with me, below their own bots
Counts against	The workspace's members limit (seats)	The bot's people per shared bot limit
Invite from	Top bar → workspace menu → Members & invites	Bot → Manage → Sharing

Only the **owner** can invite, change roles and remove people.

Roles

Role	Workspace member	Shared bot
Editor	Change any bot's settings, knowledge, appearance and domains; create bots; reply in live chat.	Change this bot's settings, knowledge, appearance and domains; reply in live chat.

Role	Workspace member	Shared bot
Viewer	Read everything: overview, conversations, sources, test retrieval.	Read this bot.

Only owners can delete bots, clear conversation history, rename the workspace and manage sharing. Viewers see a *View only* banner, and editing controls are disabled. The API enforces the same rules: a change by a viewer gets `403`, and someone without access gets `404`.

Invite someone

1. Enter their email and choose **Editor** or **Viewer**.
2. Pingie creates an invitation that's valid for **7 days** and emails it if SMTP is configured.
3. A **Copy** button shows the invite link. Send it yourself if email isn't set up.

The invited person opens the link, signs in with Google **using the invited email**, and chooses **Accept** or **Decline**. Pending invites also appear as a banner on their **Bots** page.

- Inviting an email that already has access, or already has a pending invite, is refused.
- **New link** issues a fresh link and another 7 days; the old link stops working.
- **Revoke** cancels a pending invite.
- Forwarded links don't work for anyone else: the signed-in email must match.

Manage access

From the same pages, the owner can change a person's role or remove them. Removing takes effect immediately.

People can also remove themselves:

- **Leave workspace** on the Members page,
- **Remove** on a bot card under *Shared with me*.

Switching workspaces

The workspace menu in the top bar lists every workspace you belong to, with your role in each. Your own workspace is always first and is the default. The dashboard remembers the last one you picked in this browser.

Limits

Plan	Workspace members (seats)	People per shared bot
Free	1 (just you)	3
Pro	5	Unlimited
Enterprise	Unlimited	Unlimited

Seats count current members plus pending invites, and are checked again when someone accepts. On Free you can still share individual bots. See Plans & limits.

Plans & limits

What each plan includes, what happens at a limit, and how to upgrade.

Plans

	Free	Pro	Enterprise
Price	$0	$49 / month	Custom
Bots	3	10	Unlimited
Messages / month	100	100,000	Unlimited (custom)
Sources per bot	5	25	Unlimited
Workspace members	1	5	Unlimited
People per shared bot	3	Unlimited	Unlimited
Model tiers	Fast	Fast, Balanced	Fast, Balanced, Best

	Free	Pro	Enterprise
Remove "Powered by Pingie"	—	✓	✓

The current plans and prices are always on the **Pricing** page in the dashboard. Plans are managed by the Pingie team and can change; the dashboard reads them live.

Note *Custom deals are possible: a workspace's individual limits can be raised without changing its plan. Contact support.*

What counts

- **Messages**: visitor messages and bot replies in the current calendar month (UTC), across all of a workspace's bots. Agent replies and system notes don't count.
- **Bots**: every bot in the workspace, whatever its status.
- **Sources**: top-level sources per bot. Pages inside a sitemap and individual team answers don't count separately.
- **Members**: current members plus pending workspace invites.

At a limit

Limit reached	What happens
Monthly messages	The widget shows This assistant isn't available right now, and the API returns 402 quota_exceeded until the month resets or you upgrade.
Bots	New bot becomes Upgrade to add more.
Sources	Adding a source is refused with an upgrade message.
Members / shares	New invites are refused with an upgrade message.
Model tier	Tiers outside your plan are disabled in Bot settings; the API returns 402 plan_restriction.

Usage is on the **Bots** page (message and bot meters) and on **Plan & billing**.

Upgrade

Open **Upgrade** in the top bar (or **Plan & billing**) to compare plans, then email **support@pingie.app**. The team moves your workspace to the new plan, usually within 24 hours, and the new limits apply immediately.

How Pingie works

Architecture

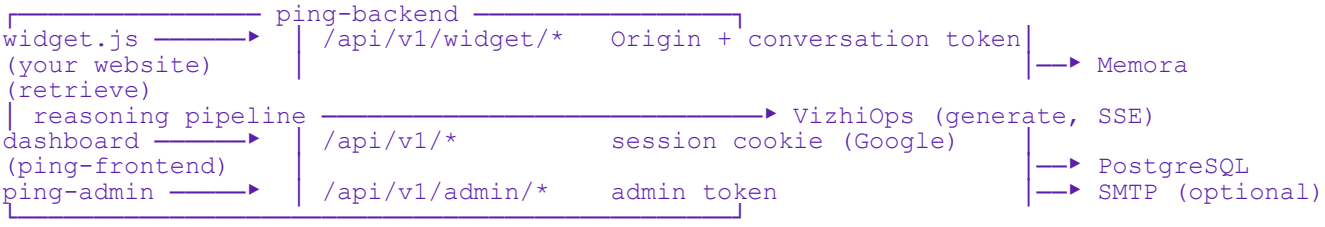
The apps that make up Pingie, the services they rely on, and how a request flows through them.

The pieces

Pingie is five small apps in one repository, plus two external services.

App	What it is	Stack
ping-backend	The API: auth, workspaces, bots, knowledge ingestion, the answer pipeline, the inbox, live chat, sharing and plans. It also serves widget.js.	Python 3.12, FastAPI, SQLAlchemy 2, Alembic, PostgreSQL (Neon)
ping-frontend	The customer dashboard.	Next.js 16 (App Router), React 19, TypeScript, plain CSS
ping-widget	The embeddable chat widget: one IIFE file, no dependencies, Shadow DOM.	TypeScript, Vite
ping-admin	The Pingie team's console for workspaces and plans.	Next.js 16
ping-doc	This documentation.	Next.js 16, Markdown

External service	Role
Memora	The retrieval engine. Pingie is one Memora application; every bot is a Memora subject and every source a Memora source. Memora does text extraction, chunking, embeddings, HNSW search, MMR diversification and reranking.
VizhiOps	The LLM gateway (OpenAI-compatible). Each model tier (fast, balanced, best) is a Vizhi model token; which model sits behind a tier is configured in Vizhi, not in Pingie.



Request paths

A visitor asks a question

1. The widget loads `GET /widget/{bot}/config`. The backend checks the page's `Origin` against the bot's allowed domains.
2. On the first message it calls `POST /widget/{bot}/conversations` and gets a conversation ID and a signed **conversation token**.
3. `POST …/messages` with `stream: true` runs the answer pipeline and streams the reply as server-sent events.
4. The reply is stored with its outcome, citations, latency, token counts and a reasoning trace. The inbox and analytics read from these rows.

Your team uses the dashboard

The dashboard calls `/api/v1/*` through its own origin (`next.config.ts` rewrites to the backend), so the session cookie is first-party and no CORS setup is needed. Every bot route resolves the bot through your membership or a share; an ID in the URL is never trusted on its own.

Knowledge ingestion

- **Files** are sent straight to Memora.
- **URLs and sitemaps** are fetched by the backend's crawler (robots.txt aware, SSRF-protected), converted from HTML to Markdown, and pushed to Memora as text.
- A background **sweeper** checks every 20 seconds which sources are still indexing and updates their progress.

Live chat

There's no websocket layer. The conversation's `status` (`bot`, `waiting`, `human`, `closed`) lives in PostgreSQL, and both the widget and the dashboard poll small "updates since" endpoints. Agent presence is a heartbeat row per user. Because the database is the source of truth, this works across any number of API processes.

Data model

```
User ──< OrganizationMember >── Organization (plan + limits)
│
└──< Bot ──< BotSource (──> Memora source)
├──< AllowedDomain
├──< BotShare >── User
├──< BotAnswer        (team answers)
└──< Visitor ──< Conversation ──< Message
└──< Lead
Plan (catalogue)    Invitation (workspace | bot)    AgentPresence (per user)
```

- **Plan limits are copied onto the organization row** when a plan is applied, so quota checks never join the plan table and a custom deal can override any single limit.
- Message rows keep `bot_id` denormalised, so analytics queries filter by bot and time without joins.
- Statuses and roles are plain strings, not database enums: adding a new value is a code change, not a migration.

Repository layout

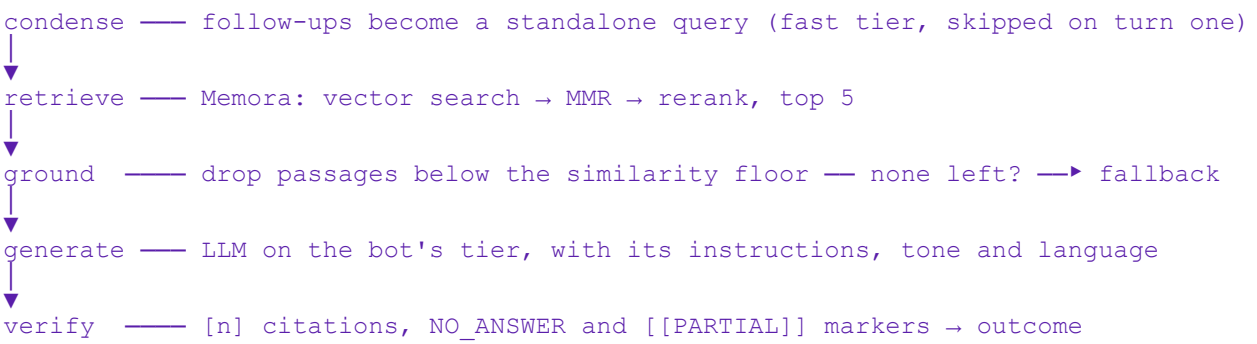
```
ping-backend/
app/
api/routes/       one router per area (widget, bots, sources, conversations, sharing,
live, …)
services/         business logic (chat_service, live_service, sharing_service,
plan_service, …)
reasoning/        the answer pipeline: prompts, output parsing, small talk
ingestion/        crawler and HTML → Markdown
clients/          Memora and VizhiOps HTTP clients
db/models/        SQLAlchemy models
migrations/         Alembic revisions
tests/              pytest, with in-memory fakes of Memora and Vizhi
ping-frontend/src/    app routes, components, lib/api.ts (typed client)
ping-widget/src/      index.ts (boot + window.Pingie), ui.ts, api.ts, storage.ts
ping-admin/src/       login, orgs, plans
ping-doc/             content/*.md, src/ (this site), scripts/export_openapi.py
```

Answer pipeline

What happens between a visitor's question and the bot's answer, and the settings that change it.

Pingie separates **finding** passages from **deciding** what to say. Memora finds candidate passages; Pingie's reasoning layer decides whether they are good enough, writes the answer with the LLM, and checks the result.

```
visitor message
│
▼
small talk? ──yes──▸ canned reply (no search, no LLM)
│ no
▼
```



1. Small talk

Greetings ("hi", "good morning", "vanakkam") and thanks ("thanks!", "perfect, thanks") get a short canned reply. This saves a search and an LLM call, and keeps them from being logged as unanswered.

2. Condense follow-ups

"What about the Pro one?" means nothing on its own. From the second message on, Pingie rewrites the latest message into a **standalone query** using the recent conversation (the last 6 turns), with the fast model tier. You can see this query in **Test retrieval**.

3. Retrieve

The standalone query goes to Memora, which searches only this bot's sources and returns the top **5** passages after diversity (MMR) and reranking. Pingie asks Memora for passages only; it never lets Memora write the answer.

4. Ground: the similarity floor

Each passage has a similarity score. Passages below the bot's **similarity threshold** (default **0.30**, set in Bot settings) are dropped. If nothing is left, the bot doesn't call the LLM at all. It replies with its **fallback message** and, if *collect email* is on, asks for the visitor's email, or offers a person when an agent is available.

Tip *Raise the threshold if the bot answers from loosely related pages. Lower it if good passages are being thrown away (check the scores in Test retrieval).*

5. Generate

The remaining passages, the conversation history and the question go to the LLM through VizhiOps, on the bot's **model tier**. The system prompt includes:

- the bot's **instructions**. The default is to answer only from the sources, keep answers under 120 words, and use numbered steps for procedures,
- its **tone**: friendly, professional or concise,
- its **reply language**: `auto` (match the visitor), English, Hindi or Tamil,
- the rule to cite passages as `[1]`, `[2]`… and to reply `NO_ANSWER` when the passages don't contain the answer.

The answer streams back token by token. Answers are capped at 700 tokens, with a low temperature (0.2) for consistency.

6. Verify

Before anything reaches the visitor, the output is filtered as it streams:

Model output	Result
Cites passages with [n]	answered. Only the passages actually cited become source chips.
Contains [[PARTIAL]]	partial. Some of the question wasn't covered.
Says NO_ANSWER	fallback. The fallback message is shown instead.

Markers are never shown to visitors. If the model declines after its first words were held back, the visitor only sees the fallback.

What's recorded

Every bot reply stores its outcome, the cited passages, a **trace** (standalone query, retrieval scores, which passages were kept and timings), latency, time to first token, token counts and the model used. This powers the inbox badges, the analytics on **Overview**, *knowledge gaps* (questions that keep falling back) and *top questions*.

Settings that affect answers

Setting	Where	Effect
Sources	Knowledge	What can be found at all.
Similarity threshold	Bot settings	How relevant a passage must be to count.
Instructions	Bot settings	Style, scope, length and persona.
Tone / reply language	Bot settings	Voice and language of answers.
Model tier	Bot settings	Speed and quality of the writing model (plan dependent).
Fallback message / collect email	Bot settings	What happens when nothing is found.
Team answers	Inbox / Sources	Exact answers for questions your docs don't cover.

Security & privacy

How Pingie authenticates people and widgets, isolates workspaces, and protects your servers and visitors.

Dashboard sign-in

- **Google is the only sign-in.** The dashboard sends Google's ID token to `POST /auth/google`. The backend verifies it with Google and checks that the token was issued for Pingie's client ID and that the email is verified.
- The backend then sets an **HttpOnly** `ping_session` cookie containing a signed JWT that's valid for 7 days. The same JWT is also accepted as `Authorization: Bearer`.
- The dashboard calls the API through its own origin, so the cookie is first-party and `SameSite=Lax`.

Access control

- Every bot route resolves the bot through **your membership of its workspace** or **a direct share**, never from the ID alone.
- **No access** returns `404` (*Bot not found*), so outsiders can't learn which IDs exist. **Too low a role** returns `403`.
- Roles rank `viewer` < `editor` < `owner`. Reads need viewer, changes need editor, and destructive or sharing actions need owner.
- Invitation links contain a random token; only its SHA-256 hash is stored. Accepting requires being signed in with the **invited email**.

The widget

The widget can't use a session: it runs on your customers' sites. Instead:

- **Origin check.** Every widget request's `Origin` header must match one of the bot's **allowed domains** (`help.acme.com` or `*.acme.com`). Localhost works only when *Allow localhost* is on, and draft bots work only on localhost.
- **Conversation token.** Starting a conversation returns a signed token bound to that conversation and bot, valid for 30 days. Every later call for that conversation must send it as `X-Conversation-Token`, so one visitor can't read or write another's chat even if a conversation ID leaks.
- **Rate limits.** Each bot accepts 20 messages per minute per visitor IP, and 60 live-chat polls per minute.
- **Monthly quota.** When the workspace's message allowance is used up, the widget stops answering (`402`).
- **Rendering.** The widget escapes all text before applying Markdown, and runs inside a Shadow DOM.

Crawling your sites

Fetching user-supplied URLs from a server is a classic SSRF risk, so the crawler:

- refuses private, loopback and link-local addresses (only a local-development setting turns this off),
- follows `robots.txt` and identifies itself as `PingieBot/1.0 (+https://pingie.app/bot)`,

- caps crawls at 200 pages, a 15-second timeout and 5 MB per page.

What's stored

Data	Purpose
Your Google name, email and avatar	Sign-in and team features.
Sources	Text and embeddings in Memora; metadata in Pingie. Deleting a source or bot deletes them in Memora too.
Conversations and messages	The inbox, analytics and live chat. Owners can clear a bot's history at any time.
Visitors	An anonymous ID from the widget, plus an email or name only if the visitor gives one or your site calls Pingie.identify(). Approximate location comes from your CDN's geo headers when present.
Leads	The email and question captured on fallback.

Admin access

The Pingie team's admin console uses a separate sign-in: Google accounts on an **allow-list**, issued a short-lived admin token signed with its own secret. A leaked dashboard secret can't mint admin tokens.

Important *If you self-host, set strong* `AUTH_JWT_SECRET` *and* `ADMIN_JWT_SECRET` *values and* `AUTH_COOKIE_SECURE=true` *in production. See Configuration.*

API

API overview

Base URL, the three ways to authenticate, error format, pagination and streaming, and the conventions every endpoint follows.

The Pingie API is a JSON REST API. The dashboard, the widget and the admin console all use it. Everything on this page applies to every endpoint in the API reference.

Base URL

```
https://api.pingie.app/api/v1
```

When self-hosting, it's your backend's origin plus `/api/v1` (locally `http://localhost:8080/api/v1`). Interactive Swagger docs are served by the backend at `/docs`.

Authentication

There are three kinds of caller, and each endpoint accepts exactly one.

Kind	Used by	How to authenticate
Dashboard session	The dashboard, scripts acting as a user	The ping_session cookie set by POST /auth/google, or the same JWT as Authorization: Bearer <token>.
Widget	The embedded widget or your own chat UI	The browser's Origin header must be on the bot's allowed domains. Conversation routes also need X-Conversation-Token, returned when the conversation is started.
Admin	ping-admin (internal)	Authorization: Bearer <admin token> from POST /admin/auth/google, for allow-listed Google accounts only.

The reference marks each endpoint with **session**, **widget**, **admin** or **public**.

Getting a session token

`POST /auth/google` takes a Google ID token (from Google Identity Services, using Pingie's OAuth client) and returns the user while setting the cookie:

```
curl -i -X POST https://api.pingie.app/api/v1/auth/google \
-H "Content-Type: application/json" \
-d '{"id_token": "<google id token>"}'
```

Sessions last 7 days. `POST /auth/logout` clears the cookie.

Errors

Errors return a JSON body with a human-readable `detail` and a stable machine-readable `code`:

```
{ "detail": "Your plan allows 3 bots. Email support@pingie.app to upgrade your plan.",
"code": "quota_exceeded" }
```

Status	code	When
400	bad_request	Malformed request.
401	unauthenticated	No session, or it's invalid or expired.
402	quota_exceeded	A plan limit was reached (messages, bots, sources, members, shares).
402	plan_restriction	The feature isn't on your plan (e.g. a model tier).

Status	code	When
403	forbidden	Signed in but not allowed: a viewer making a change, a domain not allowed for the widget, a wrong email for an invite.
404	not_found	It doesn't exist, or you have no access to it.
409	conflict	Already exists or already done (duplicate domain, pending invite, accepted invite).
422	validation_error	The body or query failed validation. detail is a list of {loc, msg} for schema errors.
429	rate_limited	Too many requests; slow down and retry.
503	service_unavailable	Retrieval or the LLM gateway is temporarily unreachable.

Conventions

- **IDs.** Bots use short public IDs like `bot_7f3k2a9`. Everything else uses UUIDs.
- **Times** are ISO 8601 in UTC, e.g. `2026-10-14T09:21:07.512Z`.
- **Pagination.** Lists that can grow (conversations) return `{ items, next_before }`. Pass `next_before` back as `?before=` to get the next page; it's `null` on the last page.
- **Partial updates.** `PATCH` changes only the fields you send.
- **Empty successes** return `204 No Content`.
- **Roles.** Reads need viewer, changes need editor, and deleting a bot, clearing history and managing sharing need owner. See Teams & sharing.

Streaming answers

`POST /widget/{bot_id}/conversations/{conversation_id}/messages` with `"stream": true` responds with `text/event-stream`:

```
event: delta
data: {"text": "Go to Settings → Billing"}

event: delta
data: {"text": " → Invoices and click Export."}

event: done
data: {"message_id": "6f0c…", "text": "Go to Settings → Billing → Invoices and click
Export.", "outcome": "answered",
"citations": [{"source_name": "Billing handbook.pdf", "location": "p. 3"}],
"asks_for_email": false, "offer_human": false, "status": "bot"}
```

- Append each `delta.text` to the visible answer, then use `done` for the stored message ID, outcome and citations.
- `EventSource` can't send a POST or custom headers, so read the stream from `fetch()`. See Build a custom chat UI.
- Without `stream`, the same `done` object is returned as plain JSON.

Rate limits

Scope	Limit
Widget messages	20 per minute per visitor IP, per bot
Widget live-chat polls	60 per minute per visitor IP, per bot
Monthly messages	Your plan's allowance, per workspace

Keeping this reference current

The API reference is generated from the backend's OpenAPI schema. After changing routes, regenerate it from `ping-doc` with `npm run gen:api`.

Build a custom chat UI

Use the public widget API directly to put Pingie inside your own interface, step by step, including streaming and live chat.

The standard widget covers most sites. If you want the chat inside your own UI (a help panel in your app, a native-looking component), call the same public **widget API** it uses. All requests go from the visitor's browser, so the `Origin` check applies. Make sure your site's hostname is in the bot's **allowed domains**.

```
const API = "https://api.pingie.app/api/v1/widget/bot_xxxxxxx";
```

1. Load the configuration

```
const config = await fetch(`${API}/config`).then((r) => r.json());
// {
//   bot_id: "bot_xxxxxxx", name: "Acme Help", status: "live",
//   online: true,              // false when paused (or draft off localhost)
//   agents_online: false,      // someone is available for live chat
//   appearance: { title, welcome, placeholder, accent, theme, position,
//                 launcher, suggestions: [...], show_sources, show_branding }
// }
```

A `403` means this origin isn't allowed, or the bot is a draft and you're not on localhost. A `404` means the bot ID is wrong.

2. Start a conversation

Create one on the visitor's first message. Keep a stable `visitor_id` per browser (8–64 characters, letters, digits, `_` or `-`), so returning visitors are recognised.

```
const visitorId = localStorage.visitorId ??= crypto.randomUUID().replaceAll("-", "");
const { conversation_id, token } = await fetch(`${API}/conversations`, {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ visitor_id: visitorId, page_url: location.href }),
}).then((r) => r.json());
```

Store both. Every later call for this conversation sends `X-Conversation-Token: <token>`. The token lasts 30 days; if you get a `401` or `404`, start a new conversation.

3. Send a message and stream the answer

```
async function ask(text, onDelta) {
const res = await fetch(`${API}/conversations/${conversation_id}/messages`, {
method: "POST",
headers: { "Content-Type": "application/json", "X-Conversation-Token": token },
body: JSON.stringify({ text, stream: true }),
});
if (!res.ok) throw new Error((await res.json()).detail);

const reader = res.body.getReader();
const decoder = new TextDecoder();
let buffer = "", done = null;
for (;;) {
const { value, done: end } = await reader.read();
if (value) buffer += decoder.decode(value, { stream: true });
let cut;
while ((cut = buffer.indexOf("\n\n")) !== -1) {
const block = buffer.slice(0, cut);
buffer = buffer.slice(cut + 2);
const event = /^event: (\w+)/m.exec(block)?.[1];
const data = JSON.parse(/^data: (.*)$/m.exec(block)?.[1] ?? "null");
if (event === "delta") onDelta(data.text);
if (event === "done") done = data;
}
if (end) break;
}
return done; // { message_id, text, outcome, citations, asks_for_email, offer_human,
status }
}
```

What to do with `done`:

Field	Use
outcome	answered, partial, fallback, smalltalk, lead, offline, or human (a person has the chat and will reply by polling; there's no bot text).
citations	Show them as sources: source_name, plus location (a URL for web pages).

Field	Use
asks_for_email	Show an email field. Send the email as the next message; the backend saves it as a lead.
offer_human	Show a "Talk to a person" button (see live chat).
message_id	Needed for feedback.

4. Feedback and identity

```
// thumbs up / down on a bot answer
await fetch(`${API}/conversations/${conversation_id}/messages/${messageId}/feedback`, {
method: "POST",
headers: { "Content-Type": "application/json", "X-Conversation-Token": token },
body: JSON.stringify({ value: "down" }),
});

// attribute the chat to a signed-in user
await fetch(`${API}/conversations/${conversation_id}/identify`, {
method: "POST",
headers: { "Content-Type": "application/json", "X-Conversation-Token": token },
body: JSON.stringify({ email: "jane@acme.com", name: "Jane Doe" }),
});
```

Live chat

When `done.offer_human` is `true`, let the visitor ask for a person:

```
const { status } = await fetch(`${API}/conversations/${conversation_id}/handoff`, {
method: "POST",
headers: { "Content-Type": "application/json", "X-Conversation-Token": token },
body: "{}",
}).then((r) => r.json());
// "waiting": an agent was online, start polling
// "unavailable": nobody online, a system message asking for the email follows
```

Then poll for agent and system messages, passing the newest `created_at` you've seen:

```
let after;
async function poll() {
const q = after ? `?after=${encodeURIComponent(after)}` : "";
const u = await fetch(`${API}/conversations/${conversation_id}/updates${q}`, {
headers: { "X-Conversation-Token": token },
}).then((r) => r.json());
// u.status: "bot" | "waiting" | "human" | "closed"
// u.agent:  { name, avatar_url } | null
for (const m of u.messages) {        // role: "agent" | "system"
render(m);                          // m.asks_for_email → show the email field
after = m.created_at;
}
const live = u.status === "waiting" || u.status === "human";
setTimeout(poll, live ? 3000 : 15000);
}
```

While the status is `waiting` or `human`, visitor messages are delivered to the agent and `POST …/messages` returns `outcome: "human"` with no text. If nobody joins within about 2 minutes, the status goes back to `bot` and a system message with `asks_for_email: true` arrives.

Tip *Poll every 3 seconds only while the chat is live and visible, and slow down or stop when the tab is hidden. The poll endpoint allows 60 requests per minute per visitor.*

Full endpoint list

See Widget (public) in the API reference.

API reference

Endpoints

Every public Pingie API endpoint, grouped by area. Paths are relative to the API origin.

Widget (public)

Method	Path	What it does
GET	/api/v1/widget/{bot_id}/config	Config. Appearance and online state. Loading this marks the domain as seen, which is what the install check reports.
POST	/api/v1/widget/{bot_id}/conversations	Start
POST	/api/v1/widget/{bot_id}/conversations/{conversation_id}/messages	Send. Ask a question. With `stream: true` the reply arrives as server-sent events: `delta` events with text, then a `done` event with the message.
POST	/api/v1/widget/{bot_id}/conversations/{conversation_id}/messages/{message_id}/feedback	Feedback
POST	/api/v1/widget/{bot_id}/conversations/{conversation_id}/identify	Identify. `Pingie.identify({email, name})`: attribute the chat to a known user.
POST	/api/v1/widget/{bot_id}/conversations/{conversation_id}/handoff	Request Handoff. The visitor asked for a person. `waiting` when an agent is online, otherwise `unavailable` and the next message is checked for an email.
GET	/api/v1/widget/{bot_id}/conversations/{conversation_id}/updates	Updates. Polled while a person is handling the chat: agent and system messages newer than `after`, plus who is answering.

Authentication

Method	Path	What it does
POST	/api/v1/auth/google	Google Login. Sign in with a Google ID token. The first sign-in creates the account and its organization.
GET	/api/v1/auth/me	Me
POST	/api/v1/auth/logout	Logout. Drop the cookie. There is no server-side session: the JWT stays valid until it expires for anyone holding a copy.

Workspaces

Method	Path	What it does
GET	/api/v1/orgs	List Orgs
GET	/api/v1/orgs/{org_id}	Get Org
PATCH	/api/v1/orgs/{org_id}	Update Org
GET	/api/v1/orgs/{org_id}/bots	List Bots

Method	Path	What it does
POST	/api/v1/orgs/{org_id}/bots	Create Bot. Create a bot (status `draft`). With `sitemap_url`, crawling starts in the background straight away.

Bots

Method	Path	What it does
GET	/api/v1/bots/{bot_id}	Get Bot
PATCH	/api/v1/bots/{bot_id}	Update Bot
DELETE	/api/v1/bots/{bot_id}	Delete Bot. Delete the bot, its conversations, and its knowledge in Memora.
GET	/api/v1/bots/{bot_id}/appearance	Get Appearance
PUT	/api/v1/bots/{bot_id}/appearance	Put Appearance
GET	/api/v1/bots/{bot_id}/overview	Overview
GET	/api/v1/bots/{bot_id}/install	Install. The embed snippet plus whether the widget has been seen on a domain.

Knowledge

Method	Path	What it does
GET	/api/v1/bots/{bot_id}/sources	List Sources. Every top-level source, with indexing progress pulled from Memora.
POST	/api/v1/bots/{bot_id}/sources/upload	Upload
POST	/api/v1/bots/{bot_id}/sources/url	Add Url. Add a page or a sitemap. Fetching happens in the background; poll the list for progress.
POST	/api/v1/bots/{bot_id}/sources/{source_id}/resync	Resync
DELETE	/api/v1/bots/{bot_id}/sources/{source_id}	Delete
POST	/api/v1/bots/{bot_id}/test	Test Retrieval. Run retrieval + reasoning for one question without storing anything.

Team answers

Method	Path	What it does
GET	/api/v1/bots/{bot_id}/answers	List Answers
POST	/api/v1/bots/{bot_id}/answers	Add Answer
DELETE	/api/v1/bots/{bot_id}/answers/{answer_id}	Delete Answer

Allowed domains

Method	Path	What it does
GET	/api/v1/bots/{bot_id}/domains	List Domains
POST	/api/v1/bots/{bot_id}/domains	Add Domain
DELETE	/api/v1/bots/{bot_id}/domains/{domain_id}	Remove Domain

Conversations & live chat

Method	Path	What it does
GET	/api/v1/bots/{bot_id}/conversations	List Conversations. Newest first, 50 per page. Pass `next_before` back as `before`.
DELETE	/api/v1/bots/{bot_id}/conversations	Clear History
GET	/api/v1/bots/{bot_id}/conversations/export.csv	Export Conversations
GET	/api/v1/bots/{bot_id}/conversations/{conversation_id}	Get Conversation. The full transcript. Opening it marks the conversation read.
GET	/api/v1/bots/{bot_id}/leads	List Leads
GET	/api/v1/bots/{bot_id}/conversations/{conversation_id}/updates	Conversation Updates. Polled by an open transcript: messages newer than `after` and the status.
POST	/api/v1/bots/{bot_id}/conversations/{conversation_id}/takeover	Take Over
POST	/api/v1/bots/{bot_id}/conversations/{conversation_id}/reply	Agent Reply
POST	/api/v1/bots/{bot_id}/conversations/{conversation_id}/handback	Hand Back
POST	/api/v1/bots/{bot_id}/conversations/{conversation_id}/close	Close

Live presence

Method	Path	What it does
POST	/api/v1/me/presence	Heartbeat. Sent every ~30 s by an open dashboard tab.
GET	/api/v1/me/live	Live. Chats waiting for a person on any bot this user can edit, and their own.

Teams & sharing

Method	Path	What it does
GET	/api/v1/orgs/{org_id}/members	List Members
POST	/api/v1/orgs/{org_id}/invitations	Invite Member
POST	/api/v1/orgs/{org_id}/invitations/{inv_id}/resend	Resend Member Invite
DELETE	/api/v1/orgs/{org_id}/invitations/{inv_id}	Revoke Member Invite
DELETE	/api/v1/orgs/{org_id}/members/me	Leave Workspace
PATCH	/api/v1/orgs/{org_id}/members/{user_id}	Set Member Role
DELETE	/api/v1/orgs/{org_id}/members/{user_id}	Remove Member
GET	/api/v1/bots/{bot_id}/shares	List Shares
POST	/api/v1/bots/{bot_id}/invitations	Invite To Bot
POST	/api/v1/bots/{bot_id}/invitations/{inv_id}/resend	Resend Bot Invite
DELETE	/api/v1/bots/{bot_id}/invitations/{inv_id}	Revoke Bot Invite
PATCH	/api/v1/bots/{bot_id}/shares/{user_id}	Set Share Role
DELETE	/api/v1/bots/{bot_id}/shares/{user_id}	Remove Share
DELETE	/api/v1/shared-bots/{bot_id}	Leave Shared Bot. Remove a bot from your own "Shared with me" list.
GET	/api/v1/shared-bots	List Shared Bots
GET	/api/v1/invitations/mine	My Invitations
POST	/api/v1/invitations/{inv_id}/accept	Accept Mine
POST	/api/v1/invitations/{inv_id}/decline	Decline Mine

Method	Path	What it does
GET	/api/v1/invitations/by-token/{token}	Preview Invite
POST	/api/v1/invitations/by-token/{token}/accept	Accept By Token
POST	/api/v1/invitations/by-token/{token}/decline	Decline By Token

Plans

Method	Path	What it does
GET	/api/v1/plans	List Public Plans

Meta

Method	Path	What it does
GET	/health	Health